

7. **Estructura de control IF.** Traducir la estructura `if (<expr>) { <codigo> }` con `(if <expr> <codigo>)`. En Lisp la estructura if sin else acaba de forma temprana con el último paréntesis. Ampliar la estructura de control if con el bloque else: `if (<expr>) { <codigo1> } else { <codigo2> }` se traduce como `(if <expr> <codigo1> <codigo2>)`. Esta ampliación puede provocar algún tipo de conflicto en el código generado por bison. Estudiad en las transparencias de teoría el origen de dicho conflicto, e intentad resolverlo. Se recomienda el uso de los marcadores de bloque `{ y }` obligatorios. Recordad que la estructura de control `if` no termina en `;` cuando se usan llaves.

Planteamos un ejemplo de estructura mínima para el if-then-else:

C	Lisp
?	<pre>(defun prueba-if (flag) (if flag 123 456))</pre>
	<pre>(print (prueba-if T)) ⇒ 123 (print (prueba-if NIL)) ⇒ 456</pre>

Esto muestra el carácter funcional de Lisp, que no tiene correspondencia en C. En Lisp las expresiones devuelven un valor. En C dichos valores deben asignarse a una variable. No se podrá usar directamente la notación funcional de Lisp al traducir desde C.

Ejemplo de traducción de C a Lisp:

C	Lisp
<pre>int a = 1 ; int b ; main () { if (a == 0) { b = 123 ; } else { b = 456 ; } printf ("%d", b) ; } //@ (main)</pre>	<pre>(setq a 1) (setq b 0) (defun main () (if (= 0 a) (setq b 123) (setq b 456)) (princ b)) (main)</pre>

Para incluir en Lisp varias sentencias en las ramas then o else del if es necesario insertar una función **progn** que reciba dichas sentencias como parámetros. Esto se ilustra en el ejemplo de función **es_par**.

<pre>int ep ; es_par (int v) { printf ("%d", v) ; if (v % 2 == 0) { puts (" es par") ; ep = 1 ; } else { puts (" es impar") ; ep = 0 ; } return ep ; }</pre>	<pre>(setq ep 0) (defun es_par (v) (princ v) (if (= (mod v 2) 0) (progn (print "es par") (setq ep 1)) (progn (print "es impar") (setq ep 0))) ep }</pre>
---	--

En C las condicionales se definen como expresiones genéricas, sin hacer distinción de si se trata de expresiones aritméticas, booleanas o de comparación. Un motivo es que en C no existe el tipo booleano, se simula mediante valores enteros. El booleano False se representa con 0, y el True con cualquier otro valor. Un problema es que en Lisp no se admiten condicionales de tipo aritmético, provocan un error. La solución inicial que vamos a adoptar **inicialmente** es suponer que los programas de prueba usarán un C de tipo canónico, **sin extravagancias como while (10-a) ...**

Ejemplos:

C	Lisp
<pre>int a = 1 ; main () { a = a && 0 ; // a = 0 ; while (10-a) { // 10-a != 0 printf ("%d", a) ; a = a + 1 ; } }</pre>	<pre>(setq a 1) (defun main () (setq a (and a 0)) (loop while (- 10 a) do ; error (princ a) (setq a (- a 1))))</pre>
<pre>int a = 1 ; main () { a = 0 ; while (a != 10) { printf ("%d", a) ; a = a + 1 ; } }</pre>	<pre>(setq a 1) (defun main () (setq a 0) (loop while (/= a 10) do (princ a) (setq a (+ a 1))))</pre>
	(main) ; Para ejecutar el programa
Imprimirá la secuencia:	0 1 2 3 4 5 6 7 8 9

Se entiende que a priori no usaremos programas en C que contengan sentencias como las marcadas en Amarillo, deberán ser como las marcadas en Cyan.

8. **Variables Locales.** Dentro del cuerpo de las funciones pueden declararse variables, que tanto en C como en Lisp serán variables locales. Se emplea la misma traducción de las globales, pero en este caso deben generarse dentro del cuerpo de la función. Hay que tener cuidado a la hora de incluir la definición de variables locales en la gramática, para evitar conflictos con la definición de las globales.

Revisad detenidamente el código del siguiente ejemplo para entender lo que se requiere en los siguientes apartados:

- Las variables globales se definirán como siempre usando **setq**
- Las variables locales se definirán usando **setq** con un cambio, concatenando su nombre con el nombre de la función. Esto se debe a que las variables definidas con **setq** son siempre variables globales en Lisp. El uso de la concatenación sirve para evitar colisiones entre variables locales y globales. Hasta el siguiente punto considerad sólo el caso de main
- Cambiad el código de salida de las sentencias de **asignación**: utilizad a partir de ahora **setf** en lugar de **setq**. Ambas funciones **setq** y **setf** son muy similares en Lisp, pero las utilizaremos para diferenciar entre declaraciones de variables con inicialización y sentencias de asignación.

C	Lisp
<pre>int a ; main () { int a = 4 ; a = a + 1 ; printf ("%d", a+1) ; } //@ (main)</pre>	<pre>(setq a 0) (defun main () (setq main_a 4) (setf main_a (+ main_a 1)) (princ (+ main_a 1))) (main) ; Para ejecutar el programa</pre>

Utilizad el guión bajo _ para concatenar nombres en lugar del guión - . El guión es demasiado ambiguo con respecto al signo unario o al operador de resta.

Para decidir qué variables utilizadas dentro de una función deben concatenarse con el nombre de la función, es necesario utilizar una tabla local en la que se insertan todas las variables declaradas localmente. Si una variable no se encuentra en esta tabla, puede deducirse que es una variable global y no debe concatenarse. Como punto de partida estudiad la función de búsqueda en la tabla de palabras reservadas.